

Beadandó Dokumentáció

AI Code Auditor – Intelligens Kódelemző Platform

Hasznos Norbert Bendegúz
DN04PZ

2025. november 23.

1 Feladat azonosítója és címe

Azonosító: A34

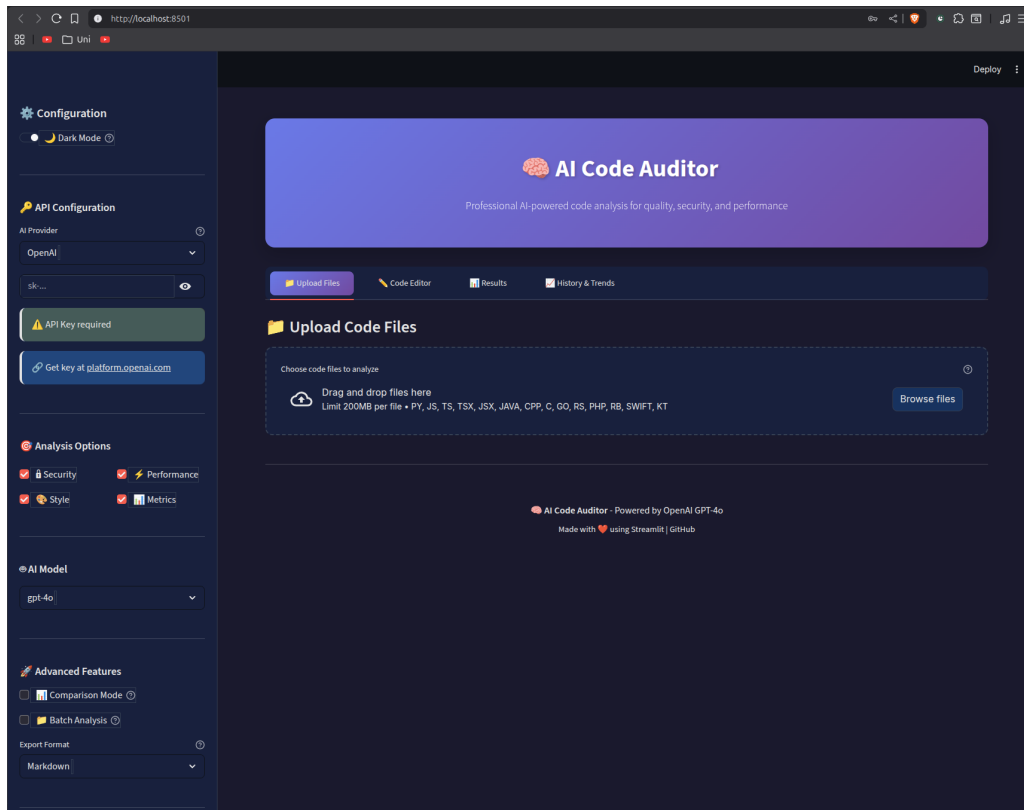
Cím: *LLM alapú kódelemző rendszer* – LLM alapú, automatizált kódminőség-ellenőrző és biztonsági auditor webalkalmazás fejlesztése.

2 A feladat megoldásának körülményei

A modern szoftverfejlesztésben a kódatvizsgálás (code review) kritikus, de időigényes folyamat. A projekt célja egy olyan grafikus felülettel rendelkező eszköz létrehozása volt, amely képes kiváltani a manuális review első körét, azonnali visszajelzést adva a fejlesztőnek a biztonsági résekről, teljesítménybeli problémákról és a kódminőségről.

A megoldáshoz a **Python** nyelvet és a **Streamlit** keretrendszert választottam, mivel ezek teszik lehetővé a leggyorsabb prototípuskészítést adatvezérelt alkalmazásokhoz. A "motorháztető alatt" a rendszer két vezető AI modellt támogat: az **OpenAI GPT-4o**-t a mélyebb elemzésekhez és a **Google Gemini**-t a költséghatékonyabb működéshez. Kiemelt figyelmet fordítottam a felhasználói élményre (Dark Mode, rezponzív UI) és a hordozhatóságra, ezért az alkalmazás teljeskörűen **Dockerizálva** lett.

A fejlesztés során sikerült megvalósítani egy robusztus rendszert, amely nemcsak elemzi a kódot, hanem vizuális statisztikákat (Plotly diagramok) is készít, valamint lehetőséget ad az eredmények PDF, JSON vagy Markdown formátumú exportálására. A *Batch Analysis* funkcióval teljes projektek átvizsgálása is megoldott.



1. ábra. Az AI Code Auditor főképernyője.

3 A feladat megoldása

A projekt teljes forráskódja, beleértve a Docker konfigurációt és a részletes telepítési útmutatót, elérhető a GitHub repository-ban.

Git Repository: <https://github.com/Fustli/ai-code-auditor>

Architektúra és Prompt Engineering

Az alkalmazás szíve a `src/code_analyzer.py` modul, amely a felhasználó által feltöltött fájlokat és a kiválasztott beállításokat (pl. Security Focus, Performance Check) egy komplex prompttá alakítja.

A megoldás során kulcsfontosságú volt a **strukturált kimenet** kikényszerítése az LLM-től, hogy a Streamlit felületén szép táblázatokat és grafikonokat tudjak megjeleníteni a nyers szöveg helyett.

```

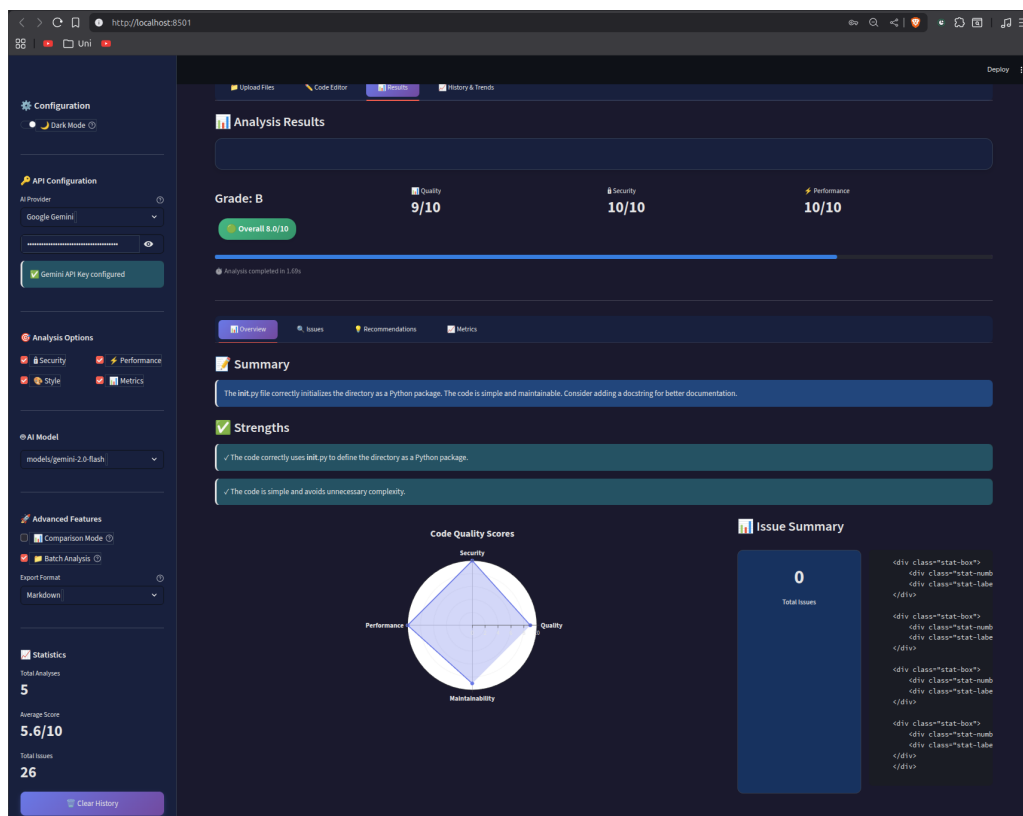
SYSTEM_INSTRUCTION = """
You are AI Code Auditor, an expert software architect.
Analyze the code based on the following weights:
- Code Quality: 40% (Readability, DRY, SOLID)
- Security: 35% (OWASP Top 10, Injection flaws)
- Performance: 25% (Complexity, Resource usage)

Output Format: Strictly valid JSON with specific keys:
"score", "grade" (A-F), "issues" (list with severity), "recommendations
".
"""

```

Listing 1. Részlet a rendszer által használt prompt logikából

A rendszer `pydantic` alapú validációt használ, hogy biztosítsa: a kapott válasz mindig feldolgozható formátumú. A frontend oldalon a `Streamlit-Ace` komponenst használtam a szintaxis-kiemelt kódszerkesztéshez, a diagramokért pedig a `Plotly` felel.



2. ábra. Elemzési eredmények: Radar diagram a kódminőségről

4 Szakmai tanulságok: Prompt Engineering

A projekt legértékesebb hozadéka a modern Prompt Engineering technikák gyakorlati elsajátítása volt. Rájöttem, hogy az "utasítsd az AI-t" megközelítés helyett strukturált mérnöki módszerekre van szükség a megbízható működéshez.

1. Context Provisioning (Kontextus biztosítása)

Megtanultam, hogy az LLM nem látja a "nagy képet", hacsak nem adjuk meg neki explicit módon. Nem elég csak a kódot beküldeni; a "Context Provisioning" során metadatákat (fájlnév, programozási nyelv, kapcsolódó config fájlok) is csatoltam a prompt-hoz. Ez drasztikusan csökkentette a tévesztéseket (pl. nem keverte össze a Java és JavaScript szintaxist hasonló kódrészleteknél).

2. Role-Based Prompting (Szerepkör alapú utasítás)

A tesztek során kiderült, hogy ha nem definiálok "személyiséget", a válaszok túl általánosak. A "Role-Based Prompting" alkalmazásával (pl. *"You are a Senior Security Engineer specialized in OWASP vulnerabilities..."*) a modell "hangszíne" megváltozott: sokkal kritikusabb, technikaibb és precízebb hibákat tárt fel, mintha csak azt írtam volna, hogy "Check this code".

3. Chain of Thought (Gondolatmenet-lánc)

A komplexebb logikai hibák megtalálásához bevezettem a "Chain of Thought (CoT)" technikát. Ahelyett, hogy azonnal a végeredményt kértem volna, arra utasítottam a modellt, hogy lépésről lépésre elemezze a kódot (pl. "First, analyze the data flow, then check for input validation..."). Ez a módszer segített feltárni olyan rejtett hibákat (pl. race condition), amiket a "zero-shot" kérések gyakran kihagytak.

4. Few-Shot Learning (Példa alapú tanulás)

A legnagyobb technikai kihívást a strukturált (JSON) kimenet garantálása jelentette a Streamlit UI számára. Itt a "Few-Shot Learning" bizonyult megoldásnak: a promptba beágyaztam 2-3 példát (bemenet -> elvárt JSON kimenet). Ezután a modell már 99%-os pontossággal tartotta be a kért sémát, így a backend hibamentesen tudta feldolgozni a választ.